

Prompt: Build SwipeX Candidate Working Area

Build the **Candidate Working Area** for my project **SwipeX – Swipe-Based Intelligent Job Discovery and Career Assistance Platform**.

The candidate working area should provide a complete job-seeker experience from profile creation and resume analysis to personalized job discovery, swiping, applying, application tracking, notifications, and analytics.

Use the uploaded SwipeX project documentation and database schema as the source of truth.

1. Technology Stack

Use:

- Frontend: React.js
- Backend: FastAPI or Django REST Framework
- Database: PostgreSQL
- Authentication: JWT
- Styling: Tailwind CSS
- API communication: Axios
- State management: Context API or Redux
- Resume/AI processing: Python-based services
- AI/NLP: OpenAI API / spaCy / sentence-transformers / scikit-learn
- File storage: Local storage during development
- API testing: Postman

Do not introduce unnecessary technologies.

2. Candidate Working Area Structure

After successful candidate login, redirect the user to:

`/candidate/dashboard`

Create the following candidate-side sections:

1. Dashboard
2. My Profile
3. Resume Management
4. AI Resume Analysis / ATS
5. Job Discovery
6. Job Search & Filters
7. Saved Jobs
8. Applications
9. Recommendations

- 10. Notifications
- 11. Analytics
- 12. Settings

Create a clean professional dashboard similar to a modern job platform, but make the **swipe-based job discovery** the main feature of SwipeX.

3. Candidate Dashboard

Create a dashboard containing:

Profile Summary

- Candidate name
- Profile completion percentage
- Current headline
- Location
- Experience
- Skills summary

Resume Summary

- Current/default resume
- Resume upload status
- ATS score
- Resume-job match score
- Resume analysis status

Job Statistics

Display cards for:

- Recommended Jobs
- Saved Jobs
- Applied Jobs
- Interviews
- Shortlisted Applications
- Rejected Applications

Recommendation Section

Show the top recommended jobs with:

- Job title
- Company
- Location
- Employment type
- Salary
- Match percentage
- Competition level

- Posted time

Provide a button:

Explore Jobs

Recent Applications

Show recent applications with:

- Job
- Company
- Applied date
- Current status

Notifications

Show the latest notifications.

4. Candidate Profile

Create a complete profile management page.

Fields should include:

- Full name
- Email
- Phone
- Profile picture
- Headline
- Summary
- Location
- Experience years
- Education
- Projects
- Certifications
- Preferred job type
- Preferred location
- Skills

Allow the candidate to:

- Add information
- Edit information
- Save changes
- View profile completion percentage

Profile information must be stored in the PostgreSQL database.

Use the `Users` and `CandidateProfile` tables.

5. Resume Management

Create a dedicated Resume Management page.

Features:

- Upload resume
- View uploaded resumes
- Set default resume
- Delete resume
- Upload multiple resume versions
- View upload date
- Display extracted skills

Supported format:

- PDF
- DOC/DOCX

After uploading a resume:

Upload Resume → Parse Resume → Extract Skills → Extract Experience → Extract Education → Extract Keywords → Store Extracted Information

Store resume information using the Resumes table.

6. AI Resume Analysis

After resume upload, provide an AI Resume Analysis section.

Display:

ATS Score

Example:

ATS Score: 82%

Resume Analysis

Show:

- Detected skills
- Detected experience
- Detected education
- Important keywords

- Missing keywords
- Missing skills
- Resume strengths
- Improvement suggestions

Job Compatibility

When a candidate selects a job, analyze:

Resume + Job Requirements

and calculate:

- ATS score
- Match percentage
- Missing skills
- Missing keywords
- Improvement suggestions

Store the result in:

ATSReports

Do not generate fake AI results. Use a clearly separated service/function for the AI analysis so it can later be connected to the actual AI model.

7. Job Discovery – Main Swipe Interface

This is the most important part of the candidate working area.

Create a Tinder-like swipe interface for jobs.

Display one job card at a time.

Each job card should contain:

- Job title
- Company
- Company type
- Location
- Employment type
- Salary range
- Experience required
- Required skills
- Posted time
- Applicant count
- Competition level
- Match percentage
- Short job description

Example:

Software Developer

ABC Technologies

Bangalore | Full-time

₹6L - ₹10L

87% Match

Low Competition

8. Swipe Actions

Implement three candidate actions:

Swipe Left

Meaning:

Skip Job

Actions:

- Record LEFT in SwipeHistory
- Move to next job
- Use the action as preference-learning data

Swipe Right

Meaning:

Interested / Apply

Actions:

- Record RIGHT in SwipeHistory
- Show job/application details
- Allow candidate to apply

Save

Meaning:

Save Job

Actions:

- Record SAVE in `SwipeHistory`
- Save the job
- Add it to Saved Jobs

The database must store:

`user_id`

`job_id`

`swipe_action`

`swiped_at`

Use the `SwipeHistory` table.

9. Personalized Recommendations

Create a recommendation engine using:

- Candidate skills
- Resume skills
- Experience
- Preferred job type
- Preferred location
- Resume-job compatibility
- Previous swipe history
- Saved jobs

Generate:

`Recommendation Score`

and:

`Recommendation Reason`

Example:

`92% Match`

`Recommended because your Python, FastAPI and PostgreSQL skills match the job requirements.`

Store recommendations using:

Recommendations

The recommendation system should learn from:

- LEFT swipes
- RIGHT swipes
- SAVE actions
- Applications

Do not implement an unnecessarily complex machine-learning system initially. Create a modular recommendation service so the algorithm can be improved later.

10. Job Search and Filtering

Create a dedicated search page.

Allow candidates to search jobs by:

- Job title
- Skills
- Company
- Location

Filters:

- MNC
- Startup
- Newly founded startup
- Remote
- Internship
- Full-time
- Salary range
- Experience level
- Fresher-friendly
- Low competition
- Recently posted

Sort options:

- Most relevant
 - Highest match
 - Recently posted
 - Lowest competition
 - Highest salary
-

11. Job Details Page

When a candidate opens a job, display:

- Job title
- Company
- Company type
- Description
- Responsibilities
- Required skills
- Experience required
- Salary
- Location
- Employment type
- Posted date
- Applicant count
- Competition level
- Match percentage
- ATS score

Buttons:

Apply Now

Save Job

Swipe Left

Swipe Right

12. Application Workflow

When the candidate clicks:

Apply Now

show an application confirmation screen.

Allow the candidate to select a resume.

Flow:

Select Resume → Review Application → Submit Application → Application Created
→ Track Application

Store:

- user_id
- job_id
- resume_id
- status
- applied_at

Use the `Applications` table.

Initial application status:

`Applied`

Possible statuses:

- Applied
- Shortlisted
- Interview
- Rejected
- Selected

13. Application Tracking

Create an Applications page.

Display applications in cards or a table.

Columns:

- Job
- Company
- Applied Date
- Resume
- Status
- Match %
- Actions

Provide filters:

- All
- Applied
- Shortlisted
- Interview
- Rejected
- Selected

Show a visual application timeline:

Applied → Shortlisted → Interview → Selected

or

Applied → Rejected

14. Saved Jobs

Create a Saved Jobs page.

Display jobs saved by the candidate.

Each card should contain:

- Job title
- Company
- Location
- Salary
- Match percentage
- Competition
- Posted date

Actions:

- View Job
 - Apply
 - Remove from Saved Jobs
-

15. Notifications

Create a notification center.

Notifications can include:

- New high-match job
- Low-competition job
- Startup hiring alert
- Application status update
- Interview update
- Recommendation update
- Resume/ATS analysis completion

Use the `Notifications` table.

Each notification should have:

- Title
- Message
- Read/unread status
- Created time

Allow:

Mark as Read

and:

Mark All as Read

16. Candidate Analytics

Create an analytics dashboard.

Display:

Application Analytics

- Total applications
- Shortlisted
- Interviews
- Rejections
- Selection rate

Swipe Analytics

- Total swipes
- Right swipes
- Left swipes
- Saved jobs

Resume Analytics

- ATS score
- Average job match percentage
- Missing skills
- Resume improvement suggestions

Recommendation Analytics

- Recommended jobs
- High-match jobs
- Applied recommended jobs

Use charts where appropriate.

Keep the analytics understandable and beginner-friendly.

17. Candidate Navigation

Create a sidebar/navbar:

```
SwipeX

Dashboard
Discover Jobs
Search Jobs
Saved Jobs
Applications
Recommendations
Resume & ATS
My Profile
Analytics
Notifications
Settings

Logout
```

Highlight the currently active page.

18. Backend API Structure

Create REST APIs approximately like:

```
/auth
/register
/login
/logout

/candidate/profile
GET
PUT

/candidate/resumes
POST
GET
DELETE

/candidate/resumes/{id}/default
PUT
```

/candidate/resumes/{id}/analyze
POST

/jobs
GET

/jobs/{id}
GET

/jobs/recommended
GET

/jobs/search
GET

/jobs/filters
GET

/swipes
POST

/saved-jobs
GET
POST
DELETE

/applications
GET
POST

/applications/{id}
GET

/recommendations
GET

/ats
POST
GET

/notifications
GET
PUT

/analytics
GET

Protect candidate APIs using JWT authentication.

Every request that accesses candidate-specific data must verify the authenticated user's `user_id`.

19. PostgreSQL Database Integration

Use the provided simplified SwipeX schema.

Main tables:

```
Users
CandidateProfile
Companies
Jobs
Resumes
Applications
SwipeHistory
ATSReports
Recommendations
Notifications
```

Respect the existing relationships.

Important relationships include:

```
Users 1 → 1 CandidateProfile
Users 1 → M Resumes
Users 1 → M Applications
Users 1 → M SwipeHistory
Users 1 → M Recommendations
Users 1 → M Notifications

Companies 1 → M Jobs

Jobs 1 → M Applications
Jobs 1 → M SwipeHistory
Jobs 1 → M ATSReports
Jobs 1 → M Recommendations

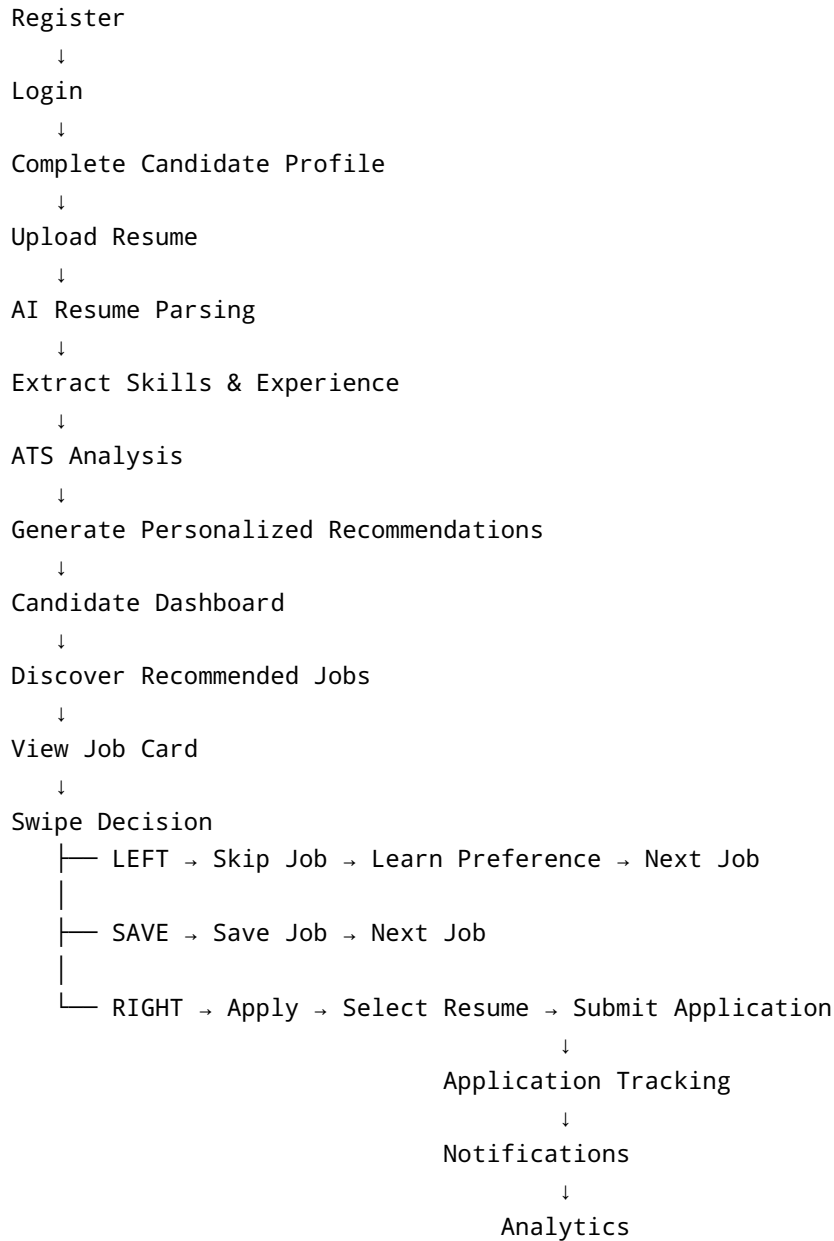
Resumes 1 → M Applications
Resumes 1 → M ATSReports
```

Do not create unnecessary tables unless they are genuinely required.

Skills, education, projects, certifications and extracted resume data can remain JSON/TEXT as specified in the simplified schema.

20. Important Candidate Flow

Implement the main workflow as:



This follows the candidate workflow supplied in the project documentation while extending it into the complete candidate working area.

21. UI/UX Requirements

Design should look like a modern professional job platform.

Requirements:

- Responsive design
- Clean dashboard
- Modern cards
- Smooth swipe animations
- Loading states
- Empty states
- Error states
- Toast notifications
- Confirmation dialogs
- Mobile-friendly layout
- Consistent spacing
- Accessible buttons and forms

The main Job Discovery page should visually emphasize the swipe card.

Use Framer Motion for smooth card/swipe animations if available.

22. Security

Implement:

- Password hashing
- JWT authentication
- Protected routes
- Role-based access
- Input validation
- File type validation
- File size validation
- Secure resume access
- SQL injection protection through ORM/query parameterization
- Environment variables for API keys
- No passwords or API keys stored in frontend code

Candidate A must never be able to access Candidate B's private profile, resumes, applications or analytics.

23. Development Approach

Build this incrementally.

Phase 1

Implement:

- Authentication

- Candidate profile
- PostgreSQL connection
- Candidate dashboard

Phase 2

Implement:

- Resume upload
- Resume management
- Resume parsing
- Skills extraction

Phase 3

Implement:

- Job listing
- Job search
- Filters
- Swipe interface
- Swipe history

Phase 4

Implement:

- ATS analysis
- Recommendation engine
- Match percentage

Phase 5

Implement:

- Applications
- Saved jobs
- Application tracking
- Notifications

Phase 6

Implement:

- Analytics
 - UI refinement
 - Error handling
 - Testing
 - Deployment preparation
-

24. Code Quality Requirements

Use a clean project structure.

Separate:

```
Frontend
  Components
  Pages
  Services
  Hooks
  Context
  Utils

Backend
  Routes
  Models
  Schemas
  Services
  AI
  Database
  Authentication
```

Do not put all logic into one file.

Use reusable components.

Keep AI/recommendation logic separate from API routes.

Use environment variables for configuration.

Add comments only where the logic is not obvious.

25. Final Requirement

Build the Candidate Working Area as a **functional full-stack module**, not just a static UI.

The final system should allow a candidate to:

```
Register/Login
  ↓
Create Profile
  ↓
Upload Resume
  ↓
```

```
graph TD; A[AI Resume Analysis] --> B[Receive Personalized Jobs]; B --> C[Swipe Jobs]; C --> D[Save / Skip / Apply]; D --> E[Track Applications]; E --> F[Receive Notifications]; F --> G[View Analytics]; G --> H[Improve Resume & Job Matching];
```

Start by implementing the database models and backend APIs, then build the React frontend and connect every screen to the real PostgreSQL-backed APIs.

Do not use mock data once the corresponding backend API has been implemented.

Keep the implementation aligned with the provided SwipeX project documentation and simplified PostgreSQL schema.